CS 305 Project One – Ava Lindgren

Document Revision History

Version	Date	Author	Comments
1.0	28 March 2026	Ava Lindgren	Initial vulnerability assessment

Client

snhu

Developer
Ava Lindgren

1. Interpreting Client Needs

Artemis Financial is a financial firm that requires a secure and modernized RESTful API to fully support its financial planning services including handling sensitive customer data (savings, investments, retirement accounts, insurance information). Because this data is highly confidential, the value of secure communications is extremely high. Encryption protocols, like HTTPS/TLS, are essential to guarantee data confidentiality and integrity during transmission between the clients and servers. Compliance with global data protection regulations (GDPR) and secure cross-border data transmission requirements is highly important as Artemis Financial can conduct international transactions and/or they support clients across different regions.

The application may be currently affected, or will be in the immediate future, due to several external threats. These external threats include injection attacks (SQL, command injection), cross-site scripting (XSS), man-in-the-middle attacks (MITM attacks), exploitation of vulnerable third-party dependencies, and distributed denial-of-service attacks (DDoS attacks).

Modernization requirements include the use of open-source libraries and frameworks like Spring Boot and embedded Tomcat. Even though these technologies do accelerate development, they do also introduce risks if dependencies are outdated or contain known vulnerabilities. Additionally, evolving web technologies and API driven architectures require secure coding practices and proper authentication mechanisms, as well as continuous monitoring for vulnerabilities.

2. Areas of Security

While referencing the vulnerability assessment process flow diagram, there are numerous areas of security that apply to Artemis Financial's software application.

Input Validation is critical since the application processes user-provided financial data, and improper validation may lead to injection attacks and data corruption.
APIs are the primary interface as the application is REST based, so securing API endpoints are essential for preventing unauthorized access in addition to data leaks.
Cryptography is relevent because sensitive financial information needs to be encrypted in both transit and rest, and improper and weak cryptographic implementations could expose this confidential information.
Client/Server is important since the distributed nature of the application, being the clients interacting with server APIs, requires secure communication and authentication mechanisms.
Code Error Handling reduces the exposure of stack traces and system details that could occur if the code is handled improperly, and attackers can use this to exploit vulnerabilities.
Code Quality, meaning secure coding practices must be used in order to reduce the likelihood of vulnerabilities like improper logic, hardcoded credentials, insecure design patterns, and more.
Encapsulation is also important as the data needs to be handled properly and with best object-oriented practices to protect sensitive data structures from unauthorized access.

3. Manual Review

Once I have manually reviewed the code in the provided code base, I have identified numerous vulnerabilities. Please see the list below.

1. Outdated Spring Boot [pom.xml]
    a. Version 2.2.4.RELEASE is used for the spring-boot-starter-parent which is outdated and not supported any longer. Many known vulnerabilities are introduced by this across transitive dependencies (Spring Core and Spring Web).
2. Outdated Dependency-Check Plugin [pom.xml]
    a. The OWASP dependency-check plugin was outdated (version 5.3.0) which I noticed immediately as this needed to be updated to run the check. Though I have updated it to its current version (12.2.0), its history of being outdated shows a history of insufficient dependency maintenance.
3. Outdated Cryptography Library (Bouncy Castle) [pom.xml]
    a. The dependency of bcprov-jdk15on (version 1.46) is significantly outdated, containing many known vulnerabilities and creating a risk in cryptographic operations like key handling and encryption.
4. Input Validation [GreetingController.java]
    a. In line 16 of the public Greeting greeting, there is no input validation logic nor is there sanitization that is needed in order to prevent injection attacks.
5. Data Access [DocData.java]
    a. In line 27, the database credentials are hardcoded directly in the code which is a great security risk that attackers can easily access. Additionally, the in-line comment reads the username and password.
6. Mishandled Errors & Exceptions [DocData.java]
    a. In line 28, the exceptions and errors are not being handled properly as they should be with secure logging, and when sensitive information is not secure, attackers may get access to the information.
7. Authorization & Authentication
    a. For all of the API controllers, there is no authentication, or access control, for any of the endpoints, and any user can have open access to these endpoints.
8. Lacking API
    a. With the general lack of any API in the code, it is not fully functioning despite its quality being quite well.
9. URL Data Access
    a. A URL is used to access the data rather than through POST method creating the possibility of the URL being exploited through it being leaked into the browser's history.

4. Static Testing

After updating the dependency-check version from 5.3.0 to 12.2.0, I ran the dependency check on Artemis Financial's software application, and once completed, the report produced an HTML output that contained the vulnerability information on March 19, 2026, at 22:18 hours. The scan identified a total of 152 vulnerabilities within 39 dependencies (21 unique), with 11 vulnerable dependencies.

These vulnerabilities were identified through the Common Vulnerabilities and Exposures Database (CVE) as well as the National Vulnerability Database (NVD)

Project: rest-service

com.twk:rest-service:0.0.1-SNAPSHOT

Scan Information (show less):

- *dependency-check version*: 12.2.0
- *Report Generated On*: Thu, 19 Mar 2026 22:18:48 -0500
- *Dependencies Scanned*: 38 (21 unique)
- *Vulnerable Dependencies*: 11
- *Vulnerabilities Found*: 152
- *Vulnerabilities Suppressed*: 0
- *NVD API Last Checked*: 2026-03-19T20:08:36-05
- *NVD API Last Modified*: 2026-03-20T00:16:18Z

Summary

Summary of All Dependencies(click to show less)

Dependency	Vulnerability IDs	Package	Highest Severity	CVE Count	Confidence	Evidence Count
bcprov-jdk15on-1.46.jar	cpe:2.3:a:bouncycastle:bouncy-castle-crypto-package:1.46:*:*:*:*:*:*:* cpe:2.3:a:bouncycastle:bouncy_castle_crypto_package:1.46:*:*:*:*:*:*:* cpe:2.3:a:bouncycastle:bouncy_castle_for_java:1.46:*:*:*:*:*:*:* cpe:2.3:a:bouncycastle:legion-of-the-bouncy-castle-java-crytography-api:1.46:*:*:*:*:*:*:* cpe:2.3:a:bouncycastle:the_bouncy_castle_crypto_package_for_java:1.46:*:*:*:*:*:*:*	pkg:maven/org.bouncycastle/bcprov-jdk15on@1.46	HIGH	4	Highest	38
classmate-1.5.1.jar		pkg:maven/com.fasterxml/classmate@1.5.1		0		54
hibernate-validator-6.0.18.Final.jar	cpe:2.3:a:hibernate:hibernate-validator:6.0.18:*:*:*:*:*:*:* cpe:2.3:a:redhat:hibernate_validator:6.0.18:*:*:*:*:*:*:*	pkg:maven/org.hibernate.validator/hibernate-validator@6.0.18.Final	MEDIUM	3	Highest	32
jackson-core-2.10.2.jar	cpe:2.3:a:fasterxml:jackson-core:2.10.2:*:*:*:*:*:*:* cpe:2.3:a:fasterxml:jackson-modules-java8:2.10.2:*:*:*:*:*:*:*	pkg:maven/com.fasterxml.jackson.core/jackson-core@2.10.2		0	Highest	43
jackson-databind-2.10.2.jar	cpe:2.3:a:fasterxml:jackson-core:2.10.2:*:*:*:*:*:*:* cpe:2.3:a:fasterxml:jackson-databind:2.10.2:*:*:*:*:*:*:* cpe:2.3:a:fasterxml:jackson-modules-java8:2.10.2:*:*:*:*:*:*:*	pkg:maven/com.fasterxml.jackson.core/jackson-databind@2.10.2	HIGH	6	Highest	39
jackson-datatype-jdk8-2.10.2.jar	cpe:2.3:a:fasterxml:jackson-modules-java8:2.10.2:*:*:*:*:*:*:*	pkg:maven/com.fasterxml.jackson.datatype/jackson-datatype-jdk8@2.10.2		0	Low	37
jakarta.annotation-api-1.3.5.jar	cpe:2.3:a:oracle:projects:1.3.5:*:*:*:*:*:*:*	pkg:maven/jakarta.annotation/jakarta.annotation-api@1.3.5		0	Low	35
jakarta.validation-api-2.0.2.jar		pkg:maven/jakarta.validation/jakarta.validation-api@2.0.2		0		56
jboss-logging-3.4.1.Final.jar		pkg:maven/org.jboss.logging/jboss-logging@3.4.1.Final		0		42
jul-to-slf4j-1.7.30.jar		pkg:maven/org.slf4j/jul-to-slf4j@1.7.30		0		26
log4j-api-2.12.1.jar	cpe:2.3:a:apache:log4j:2.12.1:*:*:*:*:*:*:*	pkg:maven/org.apache.logging.log4j/log4j-api@2.12.1	MEDIUM	2	Highest	42
log4j-to-slf4j-2.12.1.jar		pkg:maven/org.apache.logging.log4j/log4j-to-slf4j@2.12.1		0		40
logback-core-1.2.3.jar	cpe:2.3:a:qos:logback:1.2.3:*:*:*:*:*:*:*	pkg:maven/ch.qos.logback/logback-core@1.2.3	HIGH	2	Highest	31
slf4j-api-1.7.30.jar		pkg:maven/org.slf4j/slf4j-api@1.7.30		0		27
snakeyaml-1.25.jar	cpe:2.3:a:snakeyaml_project:snakeyaml:1.25:*:*:*:*:*:*:*	pkg:maven/org.yaml/snakeyaml@1.25	CRITICAL	8	Highest	44
spring-boot-2.2.4.RELEASE.jar	cpe:2.3:a:vmware:spring_boot:2.2.4:release:*:*:*:*:*:*	pkg:maven/org.springframework.boot/spring-boot@2.2.4.RELEASE	CRITICAL	3	Highest	39
spring-core-5.2.3.RELEASE.jar	cpe:2.3:a:pivotal_software:spring_framework:5.2.3:release:*:*:*:*:*:* cpe:2.3:a:springsource:spring_framework:5.2.3:release:*:*:*:*:*:* cpe:2.3:a:vmware:spring_framework:5.2.3:release:*:*:*:*:*:*	pkg:maven/org.springframework/spring-core@5.2.3.RELEASE	CRITICAL*	12	Highest	36
spring-web-5.2.3.RELEASE.jar	cpe:2.3:a:pivotal_software:spring_framework:5.2.3:release:*:*:*:*:*:* cpe:2.3:a:springsource:spring_framework:5.2.3:release:*:*:*:*:*:* cpe:2.3:a:vmware:spring_framework:5.2.3:release:*:*:*:*:*:*	pkg:maven/org.springframework/spring-web@5.2.3.RELEASE	CRITICAL*	13	Highest	34
tomcat-embed-core-9.0.30.jar	cpe:2.3:a:apache:tomcat:9.0.30:*:*:*:*:*:* cpe:2.3:a:apache_tomcat:apache_tomcat:9.0.30:*:*:*:*:*:*	pkg:maven/org.apache.tomcat.embed/tomcat-embed-core@9.0.30	CRITICAL*	49	Highest	30
tomcat-embed-el-9.0.30.jar		pkg:maven/org.apache.tomcat.embed/tomcat-embed-el@9.0.30		0		28
tomcat-embed-websocket-9.0.30.jar	cpe:2.3:a:apache:tomcat:9.0.30:*:*:*:*:*:* cpe:2.3:a:apache_tomcat:apache_tomcat:9.0.30:*:*:*:*:*:*	pkg:maven/org.apache.tomcat.embed/tomcat-embed-websocket@9.0.30	CRITICAL*	50	Highest	30

* indicates the dependency has a known exploited vulnerability

Key Vulnerabilities

Names	Brief Description	Solution
Spring Framework (spring-core, spring-web, spring-boot)	Risk of remote code execution and data exposure	Upgrade to latest stable versions
Tomcat Embedded (tomcat-embed-core, websocket)	Risk of server compromise and denial-of-service	Upgrade to secure versions
SnakeYAML (snakeyaml-1.25)	Risk of unsafe deserialization attacks	Upgrade or replace with safer alternatives
Jackson Databind (jackson-databind-2.10.2)	Risk of deserialization vulnerabilities	Upgrade to patched version
Bouncy Castle (bcprov-jdk15on-1.46)	Risk of weak cryptographic implementations	Update cryptographic library
Logback Core (logback-core-1.2.3)	Risk of logging related exploits	Upgrade dependency

5. Mitigation Plan

After the manual review and the static testing report, there are several steps to mitigate the identified security vulnerabilities. I would recommend **upgrading vulnerable dependencies** so that the critical components (like Spring, Tomcat, and SnakeYAML) are prioritized in addition to updating all third-party libraries regularly. The **implementation of strong input validation** is strongly suggested so that all user inputs are validated and sanitized, as well as using parameterized queries to prevent injection attacks. **Securing API endpoints** is crucial to enforce role-based access control and to implement authentication. **Error handling** needs to be improved to avoid the exposure of stack traces to all the users and logging detailed errors must be internal. It is essential to **remove credentials that are hardcoded** and change them to be stored in environment variables or even secure vaults. **Implementing secure logging and monitoring** is also very important as any suspicious activities should always be monitored to prevent future attacks. Additionally, regular code reviews must be conducted as a **secure development practice** as well as possibly integrating automated security testing into the CI/CD pipeline.